



FIRST WEEK ON-CALL SURVIVAL GUIDE

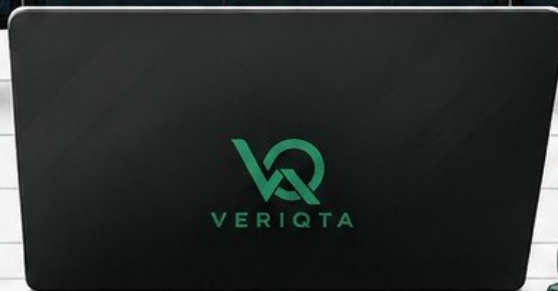
INVESTIGATE

TROUBLESHOOT

MITIGATE

COMMUNICATE

RECOVER



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

YOUR FIRST ON-CALL SHIFT: WHAT YOU ACTUALLY OWN

1 WHAT ON-CALL MEANS AT SENIOR LEVEL



- You ensure services remain reliable, secure and available.
- You respond to and manage production incidents.
- You make decisions under pressure with limited information.
- You balance speed, risk and long-term system health.
- You represent the service and own the customer impact.

2 SERVICE OWNERSHIP VS FIXING EVERYTHING



SERVICE OWNERSHIP

- Own the reliability, health and lifecycle of your service.
- Understand dependencies.
- Improve alerting, runbooks and observability.
- Drive reliability and prevent recurrence.
- Make architectural and operational improvements.



NOT THE PERSON WHO FIXES EVERYTHING

- You are not expected to fix every issue in the company.
- You are not the on-call for other teams' services.
- You do not bypass ownership boundaries.
- You escalate when the issue is outside your domain.
- You focus on your service and its dependencies.

3 WHAT YOU ARE RESPONSIBLE FOR



AVAILABILITY

Keep the service running and accessible.



CUSTOMER IMPACT

Minimize disruption and communicate clearly.



DATA INTEGRITY

Protect data from loss or corruption.



SECURITY

Detect and respond to potential security issues.



SLO RESPONSIBILITY

Meet reliability targets and manage error budgets.

4 PRIMARY VS SECONDARY ON-CALL



PRIMARY ON-CALL

- First responder.
- Acknowledges alerts.
- Investigates and mitigates.
- Drives incident resolution.
- Communicates initial updates.



SECONDARY ON-CALL

- Provides support when primary needs help.
- Brings additional expertise.
- Helps with escalation and complex issues.
- May take over if primary is unavailable.

5 INCIDENT COMMANDER VS RESPONDER



INCIDENT COMMANDER (IC)

- Owns the incident response.
- Coordinates all teams.
- Makes high-level decisions.
- Manages communication.
- Ensures clear roles and timeline.



RESPONDER (YOU)

- Investigates and provides technical findings.
- Executes mitigation steps.
- Reports status to the IC.
- Escalates when needed.
- Focuses on technical resolution within your area.

6 WHAT YOU ARE AUTHORIZED TO CHANGE



- Apply safe, reversible changes within your service.
- Use approved runbooks and standard procedures.
- Restart, scale or rollback your service.
- Modify configurations within defined boundaries.
- Toggle approved feature flags.
- Perform operational actions required for mitigation.
- Avoid changes outside your ownership or without clear approval.

7 WHEN TO INVESTIGATE, MITIGATE, ESCALATE OR HAND OVER



INVESTIGATE

When the alert is real, in your domain, and you have sufficient context to start.



MITIGATE

When you understand the impact and can take a safe action to reduce or stop it.



ESCALATE

When the issue is outside your expertise, requires another team, or is a higher severity.



HAND OVER

When another engineer or team is better placed to continue, or your shift is ending.



SENIOR ENGINEER TIP

**YOUR FIRST RESPONSIBILITY IS REDUCING IMPACT SAFELY,
NOT PROVING YOU CAN SOLVE EVERYTHING ALONE.**



@veriqta_academy



@veriqta



@veriqta



@veriqta



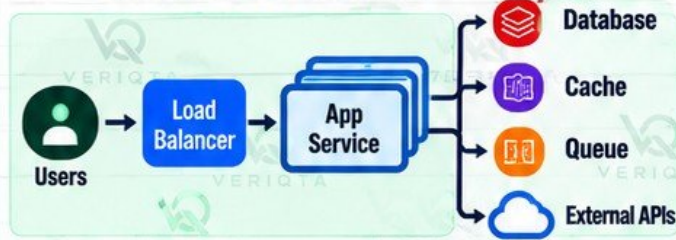
@veriqta



@veriqta

BUILD YOUR ON-CALL READINESS MAP BEFORE THE PAGER RINGS

1 SERVICES AND CRITICAL DEPENDENCIES



- List all services you own or support.
- Identify upstream and downstream dependencies.
- Understand which dependencies are critical.
- Know which failures affect customers directly.

2 PRODUCTION ENVIRONMENTS, REGIONS, CLUSTERS AND ACCOUNTS



Cloud Accounts

Production, staging, disaster recovery



Regions / AZs

Know where your services are deployed



Clusters

EKS / AKS / GKE / on-prem and their purpose



Environments

prod, staging, dev, sandbox
Know the access and context for each

THINGS TO KNOW

- Which environment is affected?
- Which region or AZ is impacted?
- Are there active failover regions?
- Do you have access to all required accounts?
- Are there any quota limitations?

3 DASHBOARDS AND OBSERVABILITY PLATFORMS



Metrics

CloudWatch, Prometheus, Grafana, Datadog



Logs

CloudWatch Logs, Loki, ELK, Splunk



Traces

X-Ray, Jaeger, Datadog APM, OpenTelemetry



Alerts

PagerDuty, Opsgenie, Alertmanager



Dashboards

Key dashboards for services and dependencies

4 LOG AND TRACE LOCATIONS



Application Logs

Location, format, filters, correlation IDs



Container / Pod Logs

kubectl logs, centralized logging



Trace Search

How to find and analyze distributed traces



System Logs

Nodes, systemd, kernel logs



Audit Logs

Cloud provider audit activity and API calls

5 RUNBOOKS AND PLAYBOOKS



- Know where runbooks are stored.
- Read them before an incident.
- Understand step-by-step procedures.
- Identify automated playbooks.
- Know when to follow a runbook vs improvise.
- Keep personal notes for gaps or improvements.

6 DEPLOYMENT HISTORY



- Check recent application deployments.
- Check infrastructure changes.
- Review configuration changes.
- Look for database migrations.
- Understand who deployed and what changed.
- Know how to roll back safely.

7 FEATURE FLAGS AND ROLLBACK MECHANISMS



- Know which features can be disabled.
- Understand feature flag systems (e.g. LaunchDarkly).
- Know rollback procedures for application and infra.
- Verify rollback time and impact.
- Identify any data or schema rollback requirements.

8 ESCALATION CONTACTS AND OWNERSHIP BOUNDARIES



- Primary and secondary on-call contacts.
- Service owners and subject matter experts.
- Team leads and managers.
- Security, compliance, and legal contacts.
- Vendor contacts for critical third-party services.
- Clear boundaries: what you own vs what you escalate.

9 CREATE A PERSONAL "WHERE DO I LOOK FIRST?" MAP



1. Check alert details and affected service.
2. Check customer impact (dashboards).
3. Check recent changes (deployments, config, feature flags).
4. Check key metrics, logs and traces.
5. Check dependent services.
6. Follow runbook or known playbook.
7. Escalate if outside your expertise or high severity.

10 PRODUCTION NOTE



DISCOVERING ACCESS PROBLEMS DURING AN INCIDENT IS ALREADY TOO LATE.

Ensure you have all required access, permissions, dashboards and tools before your on-call shift starts.

UNDERSTAND THE PRODUCTION ARCHITECTURE BEFORE TOUCHING IT

1 REQUEST PATH FROM CLIENT TO BACKEND

- Understand the full request flow from the user to all backend components.
- Know where security, routing, caching and data layers fit in.
- Identify which components are in the critical path.
- Know the dependencies for each step.

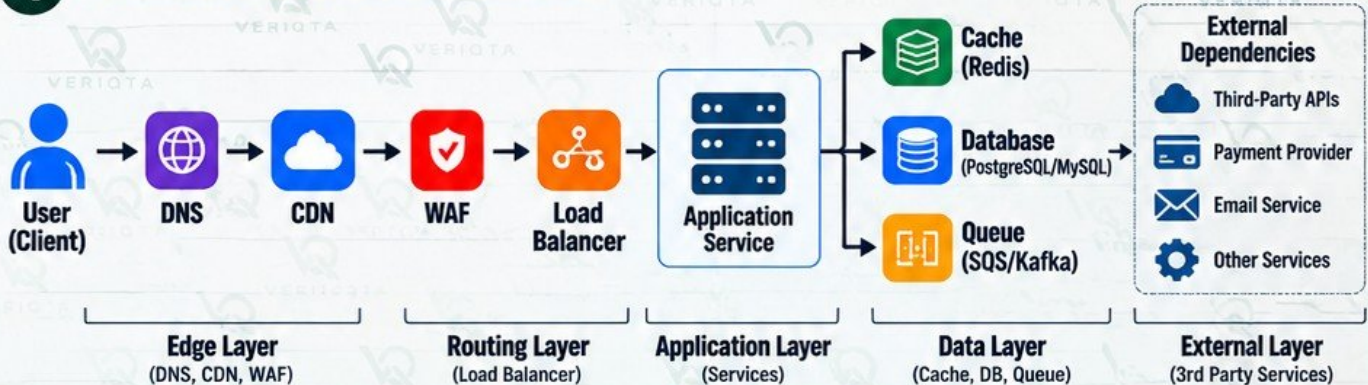
3 DNS, CDN, WAF, LOAD BALANCER, APP, CACHE, DB, QUEUE AND EXTERNAL DEPENDENCIES

- Each layer can introduce latency, failures or security risks.
- A problem in any layer can impact the whole service.
- Understand how traffic flows and how each component behaves under failure.
- Know which layers are managed and which you operate.

2 KEY COMPONENTS IN A TYPICAL ARCHITECTURE

- DNS** Resolves domain to an IP address.
- CDN** Caches content and reduces latency.
- WAF** Filters malicious traffic.
- Load Balancer** Distributes traffic to healthy instances.
- Application** Processes business logic.
- Cache** Improves performance (e.g. Redis).
- Database** Stores persistent data.
- Queue** Handles asynchronous work (e.g. SQS, Kafka).
- External Dependencies** Third-party services, APIs, etc.

4 EXAMPLE PRODUCTION ARCHITECTURE



5 CONTROL PLANE VS DATA PLANE

CONTROL PLANE

- Manages and configures the system.
- Examples: Kubernetes API server, ETCD, cloud control APIs.
- Less frequent traffic.
- Highly critical for management and orchestration.

DATA PLANE

- Handles actual user traffic.
- Examples: application services, load balancers, proxies, databases.
- High throughput and latency sensitive.
- Failures directly impact customers.

7 SYNCHRONOUS VS ASYNCHRONOUS DEPENDENCIES

SYNCHRONOUS

- Request waits for a response.
- Tightly coupled and lower latency.
- Examples: API calls, database queries.
- Failures directly impact the user request.

ASYNCHRONOUS

- Request is processed later.
- Decouples services and improves reliability.
- Examples: message queues, background workers, event streams.
- Failures are isolated and easier to recover.

9 IDENTIFY CRITICAL PATHS AND SINGLE POINTS OF FAILURE



- Map the components required for a successful user request.
- Identify single points of failure (SPOF).
- Understand where redundancy exists and where it does not.
- Know which failures will cause full or partial outage.

6 STATEFUL VS STATELESS COMPONENTS

STATEFUL

- Maintain persistent data or state.
- Examples: databases, message brokers.
- Harder to scale and recover.
- Require careful backup and replication.

STATELESS

- Do not store client state.
- Examples: web servers, API services.
- Easier to scale and replace.
- Can be freely terminated and recreated.

8 REGIONAL AND AVAILABILITY ZONE BOUNDARIES

- Know which region your services run in.
- Understand multi-AZ and multi-region design.
- Identify cross-region dependencies.
- Be aware of failover mechanisms.
- Know the impact if one AZ or region fails.

10 PRODUCTION NOTE



DISCOVERING ACCESS PROBLEMS DURING AN INCIDENT IS ALREADY TOO LATE.

Ensure you have all required access, permissions, dashboards and tools before your on-call shift starts.

KNOW THE BLAST RADIUS BEFORE YOU CHANGE ANYTHING

1 ONE USER VS MANY USERS



ONE USER

- Likely isolated issue.
- Could be client, network or specific account.
- Lower urgency.



MANY USERS

- Indicates broader impact.
- Check service health, dependencies and recent changes.
- Higher urgency.

2 ONE INSTANCE VS SERVICE-WIDE



ONE INSTANCE

- Single node or pod.
- May be automatic recovery.
- Check node health, resource pressure and restarts.



SERVICE-WIDE

- Affects all instances.
- Likely configuration, deployment, dependency or platform issue.
- Larger blast radius and urgency.

3 ONE AVAILABILITY ZONE VS REGIONAL



ONE AZ

- Only one Availability Zone affected.
- Another AZ may be healthy.
- Check AZ-level issues (network, compute, storage).



REGIONAL

- Multiple AZs or entire region affected.
- Higher customer impact.
- Could be service, network, or cloud provider issue.

4 ONE TENANT VS MULTI-TENANT IMPACT



ONE TENANT

- Isolated to a specific customer or account.
- Could be configuration, quota or data issue.



MULTI-TENANT

- Affects multiple customers.
- Likely core service or shared dependency issue.
- Higher business and reputational impact.

5 ONE ENDPOINT VS ENTIRE APPLICATION



ONE ENDPOINT

- Single API or feature.
- Other functionality may still work.
- Check recent code changes or dependency for that endpoint.



ENTIRE APPLICATION

- Most or all endpoints affected.
- Likely shared service, database, network or platform issue.
- Critical impact.

6 CONTROL PLANE VS DATA PLANE FAILURE



CONTROL PLANE

- Manages and configures the system.
- Examples: Kubernetes API server, cluster control, cloud control services.
- Outage can affect many services and operations.



DATA PLANE

- Handles actual user traffic and data.
- Examples: application pods, services, databases, caches.
- Impact may be limited to specific services.

7 INTERNAL VS CUSTOMER-FACING IMPACT



INTERNAL

- Affects internal teams only.
- Lower business impact.
- Still important, but usually less urgent.



CUSTOMER-FACING

- Affects external customers.
- Impacts revenue, trust and SLA.
- Requires faster response and clear communication.

8 DETERMINE WHETHER THE PROBLEM IS EXPANDING

CHECK CURRENT SCOPE

- How many users?
- Which services?
- Which regions/AZs?

COMPARE RECENT METRICS

- Error rate trend
- Latency trend
- Traffic trend
- Affected components

LOOK FOR SPREAD

- Is impact increasing?
- Are more services failing?
- Are new regions affected?

9 SENIOR QUESTION TO ALWAYS ANSWER



**WHAT IS BROKEN,
WHO IS AFFECTED,
AND WHAT IS STILL HEALTHY?**

- What is not working?
- Which users, services or tenants are affected?
- What parts of the system are still healthy?
- How big is the blast radius?
- Is the impact growing or stable?

10 PRODUCTION NOTE DISCOVERING ACCESS PROBLEMS DURING AN INCIDENT IS ALREADY TOO LATE.



- Ensure you have all required access, permissions, dashboards and tools before your on-call shift starts.
- Test your access regularly.
- Know escalation paths if access is unavailable.

LEARN TO READ ALERTS WITHOUT LETTING ALERTS CONTROL YOU

1 SYMPTOM ALERTS VS CAUSE ALERTS



SYMPTOM ALERTS

- Show that something is wrong.
- Triggered by visible impact (e.g. high error rate).
- Often multiple symptom alerts for the same issue.
- They tell you WHAT is happening.



CAUSE ALERTS

- Identify the underlying reason.
- Less frequent but more valuable.
- Help you fix the real issue.
- They tell you WHY it is happening.

2 WARNING VS ACTIONABLE PAGE



WARNING

- Indicates potential risk.
- Early sign of degradation.
- May not require immediate action.
- Used for visibility and trend monitoring.
- Investigate if the trend continues or worsens.



ACTIONABLE PAGE

- Requires immediate attention.
- Usually indicates real or imminent user impact.
- Has clear runbook and response steps.
- Affects availability, performance or critical functionality.

3 STATIC THRESHOLDS VS SLO-BASED ALERTING



STATIC THRESHOLDS

- Fixed value (e.g. CPU > 80%).
- Easy to set up.
- Can generate noise.
- May not reflect real user impact.
- Does not consider traffic patterns or error budget.



SLO-BASED ALERTING

- Based on Service Level Objectives (SLOs).
- Focuses on user experience (e.g. error rate, latency).
- Uses burn rates and error budgets.
- Reduces noise and increases signal quality.
- Best for production services.

4 DUPLICATE AND CORRELATED ALERTS



DUPLICATE ALERTS

- Same alert triggered multiple times.
- Often caused by multiple instances or regions.
- Can create unnecessary noise.
- Use alert grouping, deduplication or aggregation.



CORRELATED ALERTS

- Different alerts for the same underlying issue.
- Example: high latency, error rate and pod restarts.
- Look for patterns across services, regions and metrics.
- Use correlation in your observability tool.

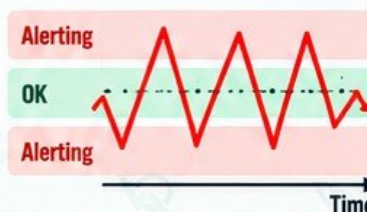
5 ALERT STORMS



ALERT STORM

- Large number of alerts in a short time.
- Usually caused by one underlying issue.
- Can hide the real problem.
- Use alert suppression, grouping, and correlation.
- Focus on key services and customer impact.

6 FLAPPING ALERTS



- Alert repeatedly fires and resolves.
- Often caused by thresholds too close to normal values.
- Can be due to transient issues or unstable metrics.
- Adjust thresholds, use sustained periods, or increase evaluation time.

7 MISSING-DATA BEHAVIOR



- Occurs when monitoring system stops receiving data.
- Can indicate real issues (e.g. service down, network problem, agent failure).
- Some tools treat missing data as a problem and alert, others do not.
- Understand how your platform handles missing data for each alert.

8 SLO BURN-RATE ALERTS



- Measures how fast you are using your error budget.
- Alerts when the burn rate is too high (e.g. 2x, 5x, 10x).
- Helps detect real user impact early.
- Uses both short and long windows (e.g. 5m and 1h).
- More effective than simple threshold alerts for critical services.

9 IS IT REAL USER IMPACT?



- Check customer-facing metrics (latency, error rate, success rate).
- Verify the blast radius (one user or many).
- Look at traffic levels and recent changes.
- Confirm across logs, traces and metrics.
- Determine if it affects external users or internal systems only.
- If no user impact, it may be a warning, not a page.

10 OPERATIONAL RISK



TREATING EVERY ALERT INDEPENDENTLY CAN HIDE ONE SHARED FAILURE.

- Multiple alerts may be symptoms of the same root cause.
- Always look for common dependencies and patterns.
- Correlate alerts before taking action.

THE FIRST FIVE MINUTES AFTER A PAGE

1 ACKNOWLEDGE THE ALERT



- Acknowledge the page in your alerting tool.
- This stops unnecessary escalations and indicates active response.
- Read the alert details, check severity and related alerts.

2 CONFIRM WHETHER THE INCIDENT IS REAL



- Check if the alert is a false positive.
- Verify using dashboards, logs or metrics.
- Look for multiple signals that confirm the issue.
- Check for maintenance windows or planned work.

3 DETERMINE CUSTOMER IMPACT



- Identify who is affected (internal, external or both).
- Check user-facing metrics (errors, latency, availability).
- Understand the business impact (revenue, critical function, SLO).
- Prioritize customer impact over internal noise.

4 ESTABLISH SCOPE



- Is it one service, multiple services or the whole platform?
- Is it one region/AZ or multi-region?
- Is it a single endpoint, tenant or all users?
- Identify the blast radius before making changes.

5 CHECK RECENT CHANGES



- Look for recent deployments.
- Check infrastructure changes (config, scaling, network, DNS, certs).
- Review feature flags and scheduled jobs.
- Correlate the timing with the alert.

6 INSPECT GOLDEN SIGNALS



- Check latency (p95, p99).
- Check error rate.
- Check traffic volume.
- Check saturation (CPU, memory, disk, etc).
- Compare with baseline to confirm anomaly.

7 CHECK DEPENDENCY HEALTH



- Verify external services (databases, queues, APIs, third-party providers).
- Check network, DNS, and load balancers.
- Look for cascading failures.
- Confirm whether the issue is upstream or downstream.

8 START AN INCIDENT TIMELINE



- Note the time the page was received.
- Record key findings, commands and actions.
- Track changes and their outcomes.
- This helps with handover, post-incident review and learning.

9 ESCALATE EARLY WHEN SEVERITY WARRANTS IT



- Escalate based on impact, not pressure.
- Involve the Incident Commander if required.
- Bring in domain experts, other teams or vendors early.
- Do not wait until the situation is out of control.
- Clear and early escalation reduces MTTR and prevents bigger outages.

DECISION CHECKLIST



- Is the alert real?
- Who is affected?
- How many users?
- Which services, regions or environments?
- What changed recently?
- Is the issue getting worse?
- Do I have enough context to mitigate?
- Who needs to be involved?

10 INVESTIGATION FLOW



SENIOR ENGINEER TIP

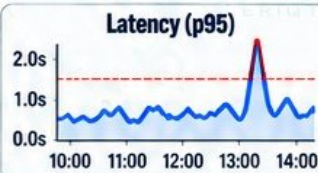
**THE GOAL IN THE FIRST FIVE MINUTES IS NOT TO SOLVE EVERYTHING.
IT IS TO UNDERSTAND THE SITUATION, REDUCE IMPACT, AND PUT THE RIGHT PEOPLE IN THE LOOP.**

GOLDEN SIGNALS: WHAT PRODUCTION IS TELLING YOU

1 LATENCY



- How long requests take to complete.
- High latency affects user experience.
- Track percentiles (p50, p95, p99) not just averages.
- Look at end-to-end latency and per-service latency.



2 TRAFFIC



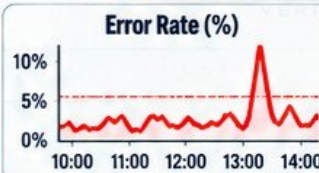
- Number of requests coming in.
- Track requests per second (RPS) or requests per minute (RPM).
- Sudden increases or drops can indicate issues.
- Compare with expected patterns (time of day, day of week).



3 ERRORS



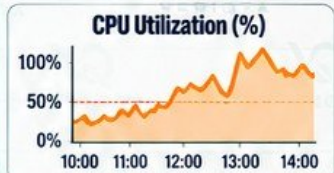
- Failed requests (4xx, 5xx).
- High error rates indicate customer impact.
- Look at error rate percentage, not just the raw count.
- Identify which endpoints or services are failing.
- Correlate with logs and traces.



4 SATURATION



- How close the system is to its limits (CPU, memory, disk, network, connections).
- High saturation can cause slowdown or failures.
- Check resource utilization across all layers.
- Look for resource contention and throttling.



5 RED METHOD RATE, ERRORS, DURATION



RATE

How many requests are coming in?



ERRORS

How many requests are failing?



DURATION

How long do requests take?

- Focuses on application-level metrics.
- Helps you quickly understand service health.
- Commonly used for web services and APIs.

6 USE METHOD UTILIZATION, SATURATION, ERRORS



UTILIZATION

How much of the resource is being used?



SATURATION

How close are we to the limit?

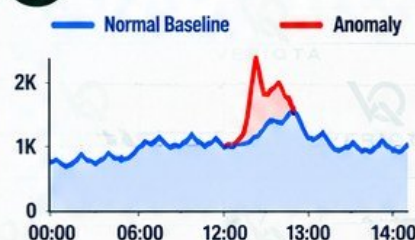


ERRORS

Are requests or operations failing?

- Focuses on infrastructure-level metrics (CPU, memory, disk, network).
- Helps identify resource bottlenecks before failure.
- Useful for hosts, containers, databases and other resources.

7 BASELINES VS ANOMALIES



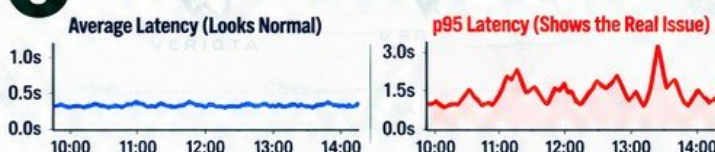
- Know what normal looks like.
- Compare current metrics with historical patterns.
- Look at time of day, day of week and seasonal trends.
- Use anomaly detection to find unusual behavior.

8 PERCENTILES (P95 AND P99)

Percentile	Meaning	Example (ms)
p50	50% of requests are faster than this	100
p95	95% of requests are faster than this	400
p99	99% of requests are faster than this	800

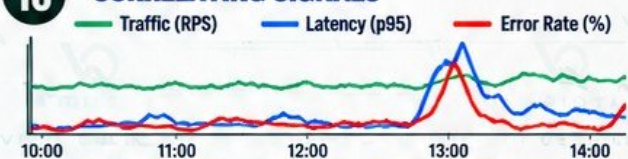
- p95 and p99 show the worst user experience.
- Averages can look fine while a small percentage of users are suffering.
- Always check percentiles for latency and duration.

9 WHY AVERAGES CAN HIDE CUSTOMER PAIN



- Averages hide spikes and bad user experience.
- A small percentage of slow requests can still impact many users.
- Always look at p95 and p99, not just the average.

10 CORRELATING SIGNALS



- Look at latency, traffic, errors and saturation together.
- A spike in traffic may explain higher latency and errors.
- Correlated signals help you find the real root cause faster.
- Do not investigate metrics in isolation.

LOGS, METRICS, TRACES, AND EVENTS:

USE THE RIGHT EVIDENCE

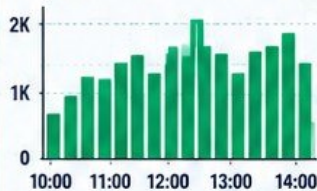
1 METRICS



ANSWER
"IS SOMETHING
CHANGING?"

- Numeric data over time.
- Shows trends, patterns and anomalies.
- Helps you detect issues early.
- Examples: request rate, error rate, latency, CPU, memory, disk, saturation.

Example: Request Rate (RPS)



2 LOGS



ANSWER
"WHAT
HAPPENED?"

- Detailed, timestamped records.
- Shows the exact events and error messages.
- Helps you understand why something happened.
- Examples: application logs, system logs, error logs.

Example: Application Log

```
2026-10-07 13:24:11 INFO [req-123]
User login successful

2026-10-07 13:24:15 ERROR [req-124]
Database connection timeout

2026-10-07 13:24:16 WARN [req-124]
Retrying in 2s...
```

3 TRACES



ANSWER
"WHERE DID THE
REQUEST SPEND
TIME?"

- Shows the full request flow across services.
- Breaks down latency per service.
- Helps you find slow components and failures.
- Examples: distributed tracing (Jaeger, Tempo, X-Ray).

Example: Trace View

Client Request	5 ms
API Gateway	12 ms
Auth Service	28 ms
User Service	320 ms
Database	45 ms
Response	5 ms

4 EVENTS



ANSWER
"WHAT
CHANGED?"

- Records system and infrastructure changes.
- Helps you correlate issues with changes.
- Examples: Kubernetes events, deployment events, cloud audit logs, configuration changes.

Example: Events

- Kubernetes event Pod restarted
- Deployment event New version deployed
- Cloud audit activity Security group updated
- Configuration change Config map modified

5 KUBERNETES EVENTS



- Pod scheduling, restarts, failures.
- Node conditions (NotReady, DiskPressure).
- Image pull errors.
- Helps identify infrastructure issues.

```
$ kubectl get events --sort-by='.lastTimestamp'
```

LAST SEEN	TYPE	REASON	OBJECT	MESSAGE
2m	Warning	FailedScheduling	pod/api	0/3 nodes
1m	Normal	Pulled	pod/api	Successfully pulled image
1m	Normal	Created	pod/api	Created container
1m	Warning	BackOff	pod/api	Back-off restarting container

6 DEPLOYMENT EVENTS



- New releases or rollbacks.
- Which version was deployed.
- Who triggered the deployment.
- Helps correlate incidents with recent changes.

Example: Deployment History

TIME	ACTION	VERSION	USER
12:05	Deployed	v1.4.2	ci-bot
11:30	Rolled back	v1.4.1	engineer
10:15	Deployed	v1.4.1	ci-bot

7 CLOUD AUDIT ACTIVITY



- API calls and configuration changes.
- Who made the change.
- When the change happened.
- Helps identify unexpected modifications.

Example: Cloud Audit Log

```
{
  "time": "2026-10-07T12:03:45Z",
  "user": "engineer@example.com",
  "action": "ModifySecurityGroup",
  "resource": "sg-123456",
  "result": "Success"
}
```

8 CONFIGURATION CHANGES



- Changes to environment variables, config maps, feature flags, or system settings.
- Can introduce unexpected behavior.
- Always check what changed recently.

Examples

- Environment variable updated
- Config map modified
- Feature flag enabled/disabled
- Certificate rotated
- DNS record changed

9 CORRELATION IDS AND REQUEST IDS



- Unique ID that follows a request across services.
- Helps you connect logs, metrics and traces.
- Essential for debugging distributed systems.

Example

Request ID: req-124
Trace ID: 4f9a8c7d2e1b...
User ID: user-5678
Include these IDs when searching logs, traces and metrics.

10 DEBUGGING INSIGHT



**EVIDENCE BECOMES
POWERFUL WHEN
ALIGNED ON THE
SAME TIMELINE.**

- Look at metrics, logs, traces and events together.
- Build a single timeline of what happened.
- Correlate changes with symptoms.
- This helps you find the real root cause faster and avoid chasing false leads.

11 PUT IT ALL TOGETHER: ONE TIMELINE



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

RECENT CHANGES:

THE HIGHEST-VALUE QUESTION IN MANY INCIDENTS

1 APPLICATION DEPLOYMENTS



- New version deployed.
- Rollback or hotfix applied.
- Check deployment time, version and who triggered it.
- Look for errors after deployment.

Where to check:

- CI/CD pipeline (e.g. GitHub Actions, Jenkins, GitLab, ArgoCD)
- Deployment dashboard
- Release notes and commit history

2 INFRASTRUCTURE CHANGES



- New resources created.
- Instances resized or replaced.
- Network or security group changes.
- Region or AZ changes.
- Infrastructure-as-Code updates (e.g. Terraform).

Where to check:

- Cloud activity logs (AWS CloudTrail, Azure Activity Log, GCP Audit Log)
- Infrastructure change pipeline
- IaC repository and history

3 CONFIGURATION CHANGES



- Application configuration updates.
- Environment variable changes.
- Config map or secret updates.
- System or OS configuration changes.

Where to check:

- Configuration management tools (e.g. Git, Ansible, Kubernetes ConfigMaps)
- Application config repository
- System change logs

4 FEATURE FLAGS



- New feature enabled or disabled.
- Gradual rollout or percentage change.
- Feature flag targeting specific users or regions.
- Unexpected interaction with existing features.

Where to check:

- Feature flag dashboard (e.g. LaunchDarkly, Unleash)
- Rollout history
- Recent flag changes and targets

5 SECRET ROTATION



- API keys or tokens rotated.
- Database credentials changed.
- TLS certificates or private keys updated.
- Some services may fail if they still use old credentials.

Where to check:

- Secrets management tool (e.g. AWS Secrets Manager, Vault)
- Rotation logs and audit trail
- Application logs for auth errors

6 CERTIFICATE CHANGES



- SSL/TLS certificate renewed or replaced.
- Certificate expired.
- Certificate chain or CA change.
- May cause service or dependency failures.

Where to check:

- Certificate management tool
- Cloud provider certificate logs
- Web server or load balancer config
- Expiry and rotation history

7 DNS CHANGES



- New DNS records created, updated or deleted.
- Changes to TTL (time to live).
- Traffic routed to a different endpoint or region.
- Propagation delays can cause intermittent issues.

Where to check:

- DNS provider (e.g. Route 53, Cloudflare, Azure DNS)
- DNS change history
- Current DNS records and TTL values

8 SCALING EVENTS



- Auto scaling triggered.
- Manual scaling changes.
- Scale in or scale out events.
- Resource limits reached (e.g. CPU, memory).

Where to check:

- Auto scaling dashboard
- Cloud provider events
- Metrics and scaling policies
- Kubernetes HPA/Cluster Autoscaler

9 DEPENDENCY RELEASES



- New version of a third-party service or library.
- Upstream API changes.
- Breaking changes or deprecated features.
- Can cause unexpected errors or performance issues.

Where to check:

- Vendor release notes
- Dependency version history
- External service status pages

10 SCHEDULED JOBS



- Batch jobs or cron jobs.
- Database maintenance tasks.
- Data processing or ETL jobs.
- Scheduled backups or reports.
- May cause high load or lock contention.

Where to check:

- Job scheduler (e.g. cron, Airflow)
- Cloud provider scheduler
- Job logs and history
- Timing and frequency

11 CORRELATION DOES NOT AUTOMATICALLY MEAN CAUSATION



- A change happening at the same time as an issue does not always mean it caused it.
- Look for supporting evidence in logs, metrics and traces.
- Consider other possible causes.
- Validate by testing or comparing patterns.
- Avoid reverting changes without evidence, as this can create additional issues.

12 BUILD A BEFORE-CHANGE VS AFTER-CHANGE TIMELINE



BEFORE CHANGE

- What was the system state?
- Metrics (latency, traffic, errors, saturation).
- Any existing issues or anomalies?
- Recent changes in dependencies?

Change Time

AFTER CHANGE

- What changed?
- How did metrics, logs and traces change?
- When did customer impact start?
- Is there a clear pattern or lag?
- Does the evidence support causation?



KEY TAKEAWAY

Build a clear timeline with changes, metrics, logs, traces and events to quickly identify what is related.

FORM HYPOTHESES INSTEAD OF RANDOMLY RUNNING COMMANDS

1 START FROM SYMPTOMS AND EVIDENCE



- Collect what you already know (symptoms, alerts, user reports).
- Look at metrics, logs, traces and recent changes.
- Define the exact problem and scope.
- Avoid jumping to solutions before understanding the issue.

Example

Symptom: High latency on checkout.
Evidence: Increased DB latency, error rate at p95, recent deploy.

2 RANK HYPOTHESES BY PROBABILITY AND IMPACT



- List possible causes.
- Estimate how likely each one is.
- Consider the potential impact if it is true.
- Focus on high-probability, high-impact hypotheses first.

Example

1. Database saturation (high impact, high probability)
2. Recent config change (medium)
3. Third-party API slowness (medium)
4. Memory leak (low probability)

3 IDENTIFY EVIDENCE THAT WOULD CONFIRM EACH HYPOTHESIS



- Define what you expect to see if the hypothesis is true.
- Check specific metrics, logs, traces or system state.
- Use clear, objective evidence.

Example

- If DB saturation: high CPU, high connections, slow queries.
- If third-party issue: high latency to external API, timeouts in traces.

4 IDENTIFY EVIDENCE THAT WOULD DISPROVE IT



- Be ready to look for evidence that proves the hypothesis is false.
- If key evidence is missing, deprioritize or discard it.
- This helps you avoid wasting time on the wrong path.

Example

- If DB CPU is normal and query latency is low, DB saturation is unlikely.
- If external API is healthy, focus elsewhere.

5 TEST THE LEAST RISKY HYPOTHESIS FIRST



- Choose a hypothesis that is safe to test.
- Start with read-only checks where possible.
- Validate your understanding before making changes.
- Minimize risk to the production environment.

Example

- Check logs and metrics before making any changes.
- Test non-disruptive diagnostics first.

6 CHANGE ONE VARIABLE AT A TIME WHERE PRACTICAL



- Make a single, controlled change.
- Observe the effect before making another change.
- This helps you understand what actually fixed the issue.
- Avoid multiple changes at once that confuse the results.

Example

- Increase one setting (e.g. connection limit).
- Wait and observe metrics before changing something else.

7 PRESERVE EVIDENCE



- Collect and save logs, metrics, traces and configurations.
- Take screenshots or export data.
- Preserve evidence before making changes that might overwrite it.
- This is important for root cause analysis and post-incident review.

Example

- Save logs before restarting a service.
- Export metrics or traces for the time window.

8 AVOID "RESTART AND HOPE"



- Restarting may temporarily hide the real issue.
- It can make troubleshooting harder and lose valuable evidence.
- Use restarts only when it is a safe, intentional mitigation step.
- Always understand why you are restarting a component.

Example

- A restart might clear memory or reopen connections, but it may not fix the root cause.
- Check metrics and logs to confirm.

9 SENIOR TROUBLESHOOTING MINDSET



- Think systematically, not desperately.
- Use data and evidence.
- Stay calm and methodical.
- Focus on reducing impact while you investigate.
- Communicate clearly with the team.

Example

"I observe the symptoms, form a hypothesis, test it safely, narrow the cause, and confirm it with evidence."

10 SENIOR TROUBLESHOOTING MODEL



KEY TAKEAWAY

A GOOD TROUBLESHOOTER DOES NOT RUN RANDOM COMMANDS. THEY USE EVIDENCE, FORM HYPOTHESES, AND TEST SYSTEMATICALLY.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

LINUX PRODUCTION TRIAGE

UNDER PRESSURE

1 CPU UTILIZATION AND LOAD

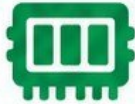


- Check overall CPU utilization.
- Look at load averages (1, 5, 15 minutes).
- Identify which processes are using CPU.
- Determine if the system is CPU bound.

Key commands:

```
$ top # real-time view
$ ps aux --sort=-%cpu | head
$ vmstat 1 5 # CPU and system activity
$ mpstat -P ALL 1 5 # per-CPU usage
```

2 MEMORY PRESSURE AND OOM BEHAVIOR



- Check memory usage and available memory.
- Look for swapping activity.
- Identify signs of OOM (out of memory).
- Find processes consuming the most memory.
- Check for memory leaks.

Key commands:

```
$ free -h # memory summary
$ top # sort by memory (Shift + M)
$ vmstat 1 5 # memory and swap activity
$ dmesg | grep -i oom # OOM events
$ cat /proc/meminfo # detailed memory info
```

3 DISK CAPACITY AND I/O



- Check disk space usage.
- Look for high disk I/O, latency or I/O wait.
- Identify full partitions.
- Check inode usage.
- Look for slow or failing disks.

Key commands:

```
$ df -h # disk space usage
$ df -i # inode usage
$ iostat -xz 1 5 # disk I/O and I/O wait
$ iotop # real-time disk I/O by process
$ lsblk -f # disk and filesystem info
```

4 FILE DESCRIPTOR EXHAUSTION



- Check current open file descriptors.
- Compare with system limits.
- Identify processes using many file descriptors.
- Look for "too many open files" errors in logs.

Key commands:

```
$ ulimit -n # current limit
$ lsof | wc -l # total open files
$ lsof -p <PID> | wc -l # per-process count
$ lsof | sort -n | tail # highest users
```

5 PROCESS STATE



- Check running, sleeping, waiting or zombie processes.
- Identify high resource processes.
- Look for stuck or defunct (zombie) processes.
- Understand process tree and relationships.

Key commands:

```
$ ps aux # all processes
$ ps -eo pid,ppid,stat,cmd # process state
$ top # interactive view
$ pstree -p # process tree
$ kill -l # available signals
```

6 NETWORK SOCKETS



- Check listening ports and services.
- Identify established connections.
- Look for high connection counts or TIME_WAIT.
- Check for network errors or drops.

Key commands:

```
$ ss -tuln # listening ports
$ ss -tan # all connections
$ ss -s # socket summary
$ netstat -s # network statistics
$ tcpdump -i any # capture traffic (if needed)
```

7 SYSTEMD SERVICE STATE



- Check service status.
- Identify failed or restarting services.
- View recent service logs.
- Check dependencies.

Key commands:

```
$ systemctl status <service> # service status
$ systemctl is-active <service> # active or not
$ systemctl list-units --failed # failed services
$ journalctl -u <service> -n 100 # service logs
$ systemctl list-dependencies <service>
```

8 KERNEL MESSAGES



- Look for kernel errors and warnings.
- Check for hardware or driver issues.
- Identify OOM, disk, filesystem or network errors.
- Look for signs of crashes or panics.

Key commands:

```
$ dmesg | tail -n 100 # recent messages
$ dmesg | grep -i error # filter errors
$ journalctl -k # kernel logs (systemd)
$ cat /var/log/kern.log # kernel log file
```

9 KEY DIAGNOSTIC TOOLS



- Use the right tool for the problem.
- Combine multiple tools for a full picture.
- Look at trends, not just snapshots.
- Correlate metrics, logs and system state.
- Always collect evidence first.

Common tools and what they help with:

Tool	Purpose
top	Real-time process and CPU view
ps	Process information
free	Memory usage
vmstat	System, CPU, memory, I/O
iostat	Disk I/O statistics
ss	Network sockets and connections
df	Disk space usage
journalctl	Systemd and service logs
dmesg	Kernel messages

10 EVIDENCE FIRST, CORRECTIVE ACTION SECOND



- Collect sufficient evidence before making changes.
- Understand the scope and impact.
- Use read-only commands where possible.
- Take screenshots or save outputs.
- Make small, reversible changes.
- Document what you did and why.



REMEMBER

- Do not make changes without understanding the evidence.
- Avoid "restart and hope".
- Validate the root cause before applying a fix.
- Corrective actions should be targeted and reversible.
- The goal is to fix the issue, not to create new ones.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

KUBERNETES PRODUCTION TRIAGE

1 POD STATUS AND RESTART COUNT



- Check pod status (Running, Pending, CrashLoopBackOff, Error, Evicted).
- Look at restart count.
- Identify pods that are not ready.
- Check how long the pod has been in its current state.

Key commands:

```
$ kubectl get pods -n <namespace>
$ kubectl get pods -n <namespace> -o wide
$ kubectl describe pod <pod-name> -n <namespace>
$ kubectl get pods -n <namespace> --sort-by=.status
.containerStatuses[0].restartCount
```

2 DEPLOYMENT AND REPLICASET HEALTH



- Check desired vs current replicas.
- Identify unavailable replicas.
- Look for rollout issues.
- Check ReplicaSet status.
- Verify if a new deployment caused the issue.

Key commands:

```
$ kubectl get deploy -n <namespace>
$ kubectl describe deploy <name> -n <namespace>
$ kubectl get rs -n <namespace>
$ kubectl describe rs <name> -n <namespace>
$ kubectl rollout status deploy <name> -n <namespace>
$ kubectl rollout history deploy <name> -n <namespace>
```

3 CONTAINER TERMINATION REASONS



- Check container state and last termination reason.
- Look at exit codes.
- Identify if the container is crashing or being killed.
- Check the time and pattern of restarts.

Key commands:

```
$ kubectl describe pod <pod-name> -n <namespace>
$ kubectl get pod <pod-name> -n <namespace> -o yaml
$ kubectl logs <pod-name> -n <namespace>
--previous
$ kubectl get events -n <namespace>
--field-selector involvedObject.name=<pod-name>
```

4 COMMON FAILURE REASONS



- OOMKilled → out of memory.
- CrashLoopBackOff → container keeps failing.
- Readiness probe failure → pod not ready to serve traffic.
- Liveness probe failure → container restarted repeatedly.

5 NODE CONDITIONS AND PRESSURE



- Check node status and conditions (Ready, NotReady).
- Look for memory, disk or PID pressure.
- Identify if the node is under resource pressure.
- Check for node taints or cordons.

Key commands:

```
$ kubectl get nodes
$ kubectl describe node <node-name>
$ kubectl get nodes -o wide
$ kubectl top nodes
$ kubectl get events --field-selector
involvedObject.kind=Node
```

6 EVENTS



- Look at recent events for pods, nodes and other resources.
- Identify scheduling issues.
- Look for image pull errors, probe failures or resource issues.
- Check the time and sequence of events.

Key commands:

```
$ kubectl get events -n <namespace>
--sort-by='.lastTimestamp'
$ kubectl describe pod <pod-name>
$ kubectl describe node <node-name>
$ kubectl get events --all-namespaces
--sort-by='.lastTimestamp'
```

7 RESOURCE REQUESTS AND LIMITS



- Check CPU and memory requests and limits.
- Look for containers hitting limits.
- Identify if the pod is being evicted due to resource pressure.
- Compare actual usage with requests and limits.

Key commands:

```
$ kubectl get pod <pod-name> -n <namespace> -o yaml
$ kubectl describe pod <pod-name> -n <namespace>
$ kubectl top pod <pod-name> -n <namespace>
$ kubectl top nodes
```

8 ROLLOUT STATE



- Check if a rollout is in progress or stuck.
- Identify failed rollouts.
- View rollout history and changes.
- Roll back if necessary.

Key commands:

```
$ kubectl rollout status deploy <name>
-n <namespace>
$ kubectl rollout history deploy <name>
-n <namespace>
$ kubectl rollout undo deploy <name>
-n <namespace>
$ kubectl get deploy <name> -n <namespace>
```

9 SERVICE ENDPOINTS AND DNS



- Check service endpoints and whether pods are healthy.
- Verify DNS resolution inside the cluster.
- Identify if the service is routing to the correct pods.
- Check for missing or failing endpoints.

Key commands:

```
$ kubectl get svc -n <namespace>
$ kubectl describe svc <service-name> -n <namespace>
$ kubectl get endpoints <service-name> -n <namespace>
$ kubectl get ep <service-name> -n <namespace>
$ kubectl exec -it <pod-name> -n <namespace>
-- nslookup <service-name>
```

10 CORE KUBERNETES TRIAGE COMMANDS



- kubectl get → list resources.
- kubectl describe → detailed information and events.
- kubectl logs → view container logs.
- kubectl get events → recent events.
- kubectl top → resource usage (pods/nodes).
- kubectl rollout status → deployment rollout status.

EXAMPLE: QUICK TRIAGE FLOW

- 1 Check pod status and events.
- 2 Look at logs and termination reason.
- 3 Verify deployment and rollout state.
- 4 Check node health and resource pressure.
- 5 Validate service endpoints and DNS.
- 6 Investigate dependencies and recent changes.

11 SENIOR ENGINEER TIP



A failing Pod may only be the visible symptom of a node, dependency, configuration, or capacity problem.

- Always look at the bigger picture.
- Correlate with recent changes.
- Check dependencies and resource usage across the cluster.
- Avoid focusing only on restarting the pod.

NETWORK AND DEPENDENCY FAILURES

1 DNS RESOLUTION



- Check if the hostname resolves correctly.
- Identify DNS server issues.
- Look for slow or intermittent DNS responses.
- Check for incorrect or stale records.
- Verify internal vs external DNS resolution.

Key commands:

```
$ nslookup example.com
$ dig example.com
$ dig +trace example.com
$ host example.com
$ cat /etc/resolv.conf
```

2 TCP CONNECTION ESTABLISHMENT



- Check if you can establish a TCP connection.
- Identify firewall or security group issues.
- Verify the correct port is open.
- Look for connection drops or resets.
- Check for ephemeral port exhaustion.

Key commands:

```
$ nc -zv example.com 80
$ telnet example.com 80
$ ss -tln
$ netstat -an | grep :80
$ timeout 5 bash -c '</dev/tcp/example.com/80'
```

3 TLS HANDSHAKE



- Verify the TLS handshake completes successfully.
- Check certificate validity, expiry and trust chain.
- Identify protocol or cipher mismatch.
- Look for SNI or hostname mismatches.
- Detect TLS handshake timeouts or errors.

Key commands:

```
$ openssl s_client -connect
example.com:443 -servername example.com
$ curl -Iv https://example.com
$ nmap --script ssl-enum-ciphers
-p 443 example.com
```

4 CONNECTION REFUSED VERSUS TIMEOUT



Connection Refused

- Service is reachable but not listening.
- Port is closed.
- Service may be down or misconfigured.



Timeout

- No response at all.
- Network, firewall or routing issue.
- Packet is not reaching the destination.
- Could also indicate dependency saturation.

5 ROUTING FAILURES



- Check if the traffic is routed to the correct network and destination.
- Look for missing or incorrect routes.
- Verify public and private routing (VPC, subnet, on-prem).
- Check for asymmetric routing or blackholes.
- Validate that NAT, gateways and peering are healthy.

Key commands:

```
$ ip route
$ traceroute example.com
$ mtr example.com
$ curl --connect-timeout 5 https://example.com
$ ip a
```

6 LOAD BALANCER HEALTH



- Check load balancer target health status.
- Verify health check configuration.
- Look for unhealthy or draining targets.
- Identify cross-zone or regional issues.
- Check if the load balancer is returning errors or timing out.

Key commands:

```
$ kubectl get svc -n <namespace>
$ kubectl describe svc <service-name>
$ aws elbv2 describe-target-health
--target-group-arn <arn>
$ curl -I https://<load-balancer-dns>
```

7 CONNECTION POOL EXHAUSTION



- Too many open connections to a dependency.
- New requests cannot get a connection.
- Look for slow queries or unreleased connections.
- Check database or service connection limits.

Key commands:

```
$ ss -s
$ ss -tan | grep <service-port>
$ netstat -an | grep ESTABLISHED | wc -l
$ lsof -i:<port> | wc -l
```

8 PACKET LOSS AND LATENCY



- Check for high latency between services.
- Look for packet loss.
- Identify intermittent connectivity issues.
- Use multiple locations to test (client, server, external).
- Determine if the issue is network or service-side.

Key commands:

```
$ ping example.com
$ mtr example.com
$ traceroute example.com
$ curl -w "time_connect: %{time_connect}
time_total: %{time_total}\n" -o /dev/null -s
https://example.com
```

9 DEPENDENCY SATURATION



- Downstream service is at capacity.
- High latency or error rates from the dependency.
- Rate limits being exceeded.
- Look for queue buildup.
- Check CPU, memory or connection limits on the dependency.

Key commands:

```
$ kubectl top pods -n <namespace>
$ curl -s -o /dev/null -w "%(http_code)\n"
https://dependency-endpoint
$ kubectl describe hpa <name>
$ kubectl get endpoints <service-name>
```

10 UPSTREAM VERSUS DOWNSTREAM FAILURES



- Upstream failures: client cannot reach your service or sends bad requests.
- Downstream failures: your service cannot reach a dependency or gets errors/timeouts.
- Use metrics, logs and traces to identify which side is failing.

11 DEBUG FLOW



SENIOR ENGINEER TIP

A failing Pod may only be the visible symptom of a node, dependency, configuration, or capacity problem. Always investigate the full path and the dependencies.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

RETRIES, TIMEOUTS, AND CASCADING FAILURE

1 WHY RETRIES CAN INCREASE AN OUTAGE



- Retries add more load to an already struggling system.
- If the dependency is slow, retries can make it slower or completely unavailable.
- Many clients retry at the same time, turning a small issue into a large outage.

Key takeaway:

- Retries are not free. They can amplify an existing problem.
- Always understand the impact before enabling retries.

2 RETRY AMPLIFICATION



- Multiple layers (client, load balancer, service) may all retry the same request.
- This can multiply the traffic significantly.
- A small number of failing requests can turn into a large volume of retries.
- It can saturate the dependency and the entire system.

3 TIMEOUT MISMATCH



- Different services often have different timeout values.
- If the upstream timeout is longer than the downstream, requests may be retried unnecessarily.
- This can lead to wasted resources and confusing errors.
- Timeouts should be aligned across services.

4 RETRY STORMS



- When many clients retry at the same time, it creates a retry storm.
- This can cause a sudden spike in traffic.
- It can overwhelm the dependency and other related services.
- Often happens during partial outages or high latency events.

5 CONNECTION EXHAUSTION



- Retries can open many more connections.
- This can exhaust the connection pool on the client or server.
- It can also exhaust ephemeral ports, leading to failures.
- Connection exhaustion can cause timeouts and new requests to fail.

6 THREAD AND WORKER EXHAUSTION



- Each request uses a thread, worker or async resource.
- Retries can consume all available workers.
- When workers are exhausted, new requests cannot be processed.
- This can cause the entire service to become unresponsive.

7 EXPONENTIAL BACKOFF



- Wait for an increasing amount of time between retries.
- Reduces load on the dependency.
- Gives the system time to recover.
- Common pattern: 1s, 2s, 4s, 8s, etc.

8 JITTER



- Adds randomness to the retry delay.
- Prevents all clients from retrying at the same time.
- Reduces the chance of a retry storm.
- Often used with exponential backoff.

9 CIRCUIT BREAKERS



- Stop sending requests to a failing dependency.
- Fail fast instead of continuously retrying.
- Allow the system to recover.
- Prevent cascading failures across services.

10 LOAD SHEDDING



- Intentionally reject or throttle requests when the system is under high load.
- Protects critical services and dependencies.
- Helps keep the system stable.
- Can be based on metrics such as CPU, latency or queue length.

11 RETRY BUDGETS



- Limit the total number of retries allowed.
- Prevent unlimited retry behavior.
- Helps control the impact on the dependency.
- Can be based on a percentage of normal traffic.



SENIOR ENGINEER TIP

- Use retries, timeouts, backoff and circuit breakers together.
- Understand the behavior of your entire request path.
- Monitor dependencies and set sane limits.
- Always test failure scenarios before they happen in production.

12 FAILURE FLOW



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

MITIGATE FIRST, ROOT-CAUSE SECOND

1 STABILIZATION VERSUS PERMANENT CORRECTION



- Stabilization stops the immediate impact and restores service.
- Permanent correction addresses the underlying root cause.
- You often need both, but stabilization always comes first.
- Communicate clearly which actions are temporary.

Example

- Temporarily scale the service to restore availability, then fix the memory leak.
- Disable a problematic feature now, then release a proper code fix.

2 ROLLBACK



- Rollback to a previous known good version.
- Use deployment history to find a stable release.
- Verify that the rollback actually resolves the issue.
- Monitor closely after rollback for side effects.

Example

```
$ kubectl rollout undo deployment <name>
$ kubectl rollout history deployment <name>
$ helm rollback <release> <revision>
```

3 TRAFFIC SHIFTING



- Shift traffic away from the failing component.
- Route traffic to a healthy region or environment.
- Use load balancer weights or service mesh routing.
- Gradually shift traffic back after the issue is resolved.

Example

- Route traffic to a healthy region.
- Use feature flags or service mesh rules to shift a percentage of traffic.

4 DISABLING A FEATURE



- Turn off a non-critical or problematic feature.
- Use feature flags for quick disablement.
- Reduces load and isolates the impact.
- Communicate the change to stakeholders.

Example

- Disable a new feature that is causing high error rates.
- Use a feature flag to turn off a customer-facing feature temporarily.

5 SCALING CAPACITY



- Increase replicas or instances to handle higher load.
- Scale at the application, node or cluster level.
- Useful when the service is healthy but under heavy load.
- Continue monitoring to confirm it stabilizes the issue.

Example

- Scale a deployment from 3 to 10 replicas.
- Add more worker nodes to the cluster.
- Increase instance size temporarily.

6 ISOLATING UNHEALTHY DEPENDENCIES



- Identify the failing dependency.
- Isolate it by disabling calls, using a fallback or cache.
- Prevent the failure from spreading to your service.
- Use timeouts, circuit breakers or feature flags.

Example

- Return cached data when the database is slow.
- Disable calls to a failing third-party API.

7 RATE LIMITING



- Limit the number of requests per client or source.
- Protect your service from overload.
- Helps keep the system stable during an incident.
- Return appropriate error responses (e.g. 429).

Example

- Use an API gateway or load balancer to enforce rate limits.
- Apply stricter limits during an ongoing incident.

8 LOAD SHEDDING



- Intentionally reject or drop non-critical requests when under high load.
- Prioritize critical traffic and services.
- Keeps the core functionality available.
- Use clear error responses and monitoring.

Example

- Drop low-priority background requests.
- Disable non-essential endpoints.
- Return a friendly error message to clients.

9 FAILOVER



- Switch traffic to a healthy standby or secondary system.
- Used for multi-region or high-availability setups.
- Can be automatic or manual.
- Validate that the failover target is healthy.

Example

- Fail over to a secondary region.
- Switch to a standby database.
- Use DNS or load balancer failover.

10 WHEN RESTARTING IS JUSTIFIED



A restart can be justified when:

- The service is stuck or in a bad state.
 - There is a memory leak or resource issue.
 - A configuration change requires it.
- It should not be the first step without evidence.

Example

- Restart a service that is unresponsive.
- Restart after applying a configuration fix.

11 REVERSIBLE VERSUS IRREVERSIBLE ACTIONS



- Prefer reversible actions during an incident.
- Understand the impact before making irreversible changes.
- Document what you changed and why.
- Be extra careful with data changes or deletions.

Example

- Reversible: scaling, feature flag, rollback.
- Irreversible: deleting data, schema changes.

12 ARCHITECTURE TRADE-OFF



- The fastest mitigation is not always the safest mitigation.
- A quick fix may introduce new risks or hidden impact.
- Consider availability, consistency, cost and long-term effects.
- Choose a solution that restores service without creating bigger problems.

Example

- Disabling a feature is fast but may impact revenue.
- A controlled rollout or traffic shift may be slower but safer.



KEY TAKEAWAY

MITIGATE FIRST TO RESTORE SERVICE, THEN INVESTIGATE AND FIX THE ROOT CAUSE.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

PRODUCTION CHANGES DURING AN INCIDENT

1 ESTABLISH A CHANGE OWNER



- Assign a clear owner for every change during an incident.
- The owner is responsible for planning, executing and monitoring the change.
- Avoid multiple people making changes at the same time.

Example:

- Change Owner: David
- Approver (if required): On-call Lead
- Record the change in the incident channel.

2 STATE THE HYPOTHESIS BEFORE CHANGING ANYTHING



- Be clear about what you think is causing the issue.
- Write the hypothesis in simple terms.
- This helps you choose the right change and measure the outcome.
- It prevents random changes that can make things worse.

Example:

- Hypothesis: High memory usage is causing the application to restart.

3 RECORD THE EXACT CHANGE



- Document the exact command or configuration change.
- Include who made the change, when it was made, and why.
- Record it in the incident channel, ticket, or change log.
- This makes it easy to review and roll back if needed.

Example:

- 2026-10-01 14:22 UTC – Increased replicas from 2 to 4 using: `kubectl scale ...`
- Reason: Application latency is high.

4 DEFINE EXPECTED OUTCOME



- Be specific about what should happen after the change.
- Define how you will know it worked.
- Use measurable indicators (e.g., error rate, latency, CPU, memory, availability).

Example:

- Expected outcome: Error rate decreases to < 1% within 5 minutes.
- API response time returns to < 300ms.

5 DEFINE ROLLBACK CRITERIA



- Decide in advance when to roll back.
- Use clear and measurable conditions.
- Do not wait too long if the change is making things worse.
- Have the rollback command ready before making the change.

Example:

- Rollback if error rate increases above 5%.
- Rollback if latency is higher after 5 minutes.
- Rollback command: `kubectl rollout undo ...`

6 PREFER REVERSIBLE ACTIONS



- Use changes that can be easily reversed.
- Prefer configuration changes, scaling changes, or feature flags.
- Avoid changes that are difficult or risky to undo.
- Irreversible changes can increase downtime and risk.

Example:

- Prefer: scale, config change, feature flag.
- Avoid: deleting data, schema changes, destructive operations.

7 AVOID SIMULTANEOUS UNCONTROLLED CHANGES



- Do not make multiple changes at the same time.
- Change one thing at a time where possible.
- This helps you understand what caused the improvement or the problem.

Example:

- Change only one configuration or setting.
- Wait and observe before making another change.

8 WATCH RELEVANT SIGNALS AFTER EXECUTION



- Monitor the key metrics, logs, and traces after the change.
- Compare with the expected outcome.
- Look for both improvements and unexpected side effects.
- Give the change enough time to take effect.

Example signals:

- Error rate, latency, CPU, memory, request count.
- Application logs and health checks.
- Downstream dependency health.

9 STOP WHEN EVIDENCE CONTRADICTS THE HYPOTHESIS



- If the change does not produce the expected outcome, stop.
- Do not keep making changes hoping it will work.
- Re-evaluate the hypothesis with the new evidence.
- Consider rolling back and trying a different approach.

Example:

- If error rate is higher after the change, stop and roll back.
- Update the hypothesis based on new evidence.

10 CHANGE LOOP



HYPOTHESIS

Identify the likely cause and expected outcome.



CHANGE

Make a small, controlled change.



OBSERVE

Monitor metrics, logs and behavior.



VERIFY OR ROLLBACK

Confirm improvement or roll back if needed.



ARCHITECTURE TRADE-OFF

Fast, reversible changes reduce risk during an incident. The goal is to restore service safely, then find and fix the real cause.



SENIOR ENGINEER TIP

The fastest mitigation is not always the safest mitigation. Always consider the long-term impact of the change.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

INCIDENT COMMUNICATION

WITHOUT CREATING MORE CHAOS

1 DECLARE SEVERITY CLEARLY



- Use a clear and consistent severity scale (e.g. SEV1-SEV4).
- State the severity early and update if it changes.
- Base severity on customer impact, not internal noise.
- Be honest and realistic.

Example:

SEV1 - Major customer impact. Users are unable to access the service. We are investigating.

2 STATE KNOWN FACTS SEPARATELY FROM ASSUMPTIONS



- Clearly separate what is confirmed from what is assumed.
- Label assumptions as "assuming" or "possible".
- Update or remove assumptions as you learn more.
- Avoid presenting assumptions as facts.

Example:

Facts: API is returning 500 errors since 14:05 UTC.
Assumption: This may be related to the recent deployment (investigating).

3 COMMUNICATE CUSTOMER IMPACT



- Be clear about who is affected and what they are experiencing.
- Explain the impact in simple, non-technical language.
- Update when the impact changes.
- Avoid minimizing or overstating the impact.

Example:

Customers are currently unable to log in to the web application. Other services are working as normal.

4 ASSIGN OWNERSHIP



- Assign a clear incident lead and owners for key workstreams.
- Make sure everyone knows their responsibilities.
- Update ownership if people rotate or need support.
- Avoid multiple people making conflicting decisions.

Example:

Incident Lead: Alex
API Investigation: Priya
Infrastructure: David
Communications: Sam

5 MAINTAIN A TIMESTAMPED TIMELINE



- Record key events, decisions and changes with timestamps (UTC).
- Include what was observed, what was done, and the result.
- Keep the timeline accurate and up to date.
- Use it for post-incident review.

Example:

14:05 - Alert triggered (high error rate)
14:07 - Confirmed customer impact
14:12 - Rolled back deployment
14:15 - Error rate decreasing

6 AVOID SPECULATIVE ROOT-CAUSE STATEMENTS



- Do not state a root cause until it is confirmed.
- Avoid speculation about blame.
- Focus on facts, investigation and next steps.
- It is okay to say "we do not know yet".

Example:

Avoid: "The database caused this."

Use: "We are investigating a possible database issue."

7 PROVIDE CONCISE STATUS UPDATES



- Send regular, concise updates even if there is no major change.
- Include current status, impact, what is being done, and next update time.
- Keep updates short and focused.
- Use a consistent channel (e.g. Slack, Status page, email).

Example:

Status Update (14:30 UTC):

- Impact: Users still unable to log in
- Current status: Rollback completed, monitoring
- Next update: In 15 minutes

8 KNOW WHEN OTHERS MUST BE INVOLVED



- Escalate to leadership for major customer or business impact.
- Involve security for potential security incidents or data exposure.
- Engage vendors or third-party providers when they are part of the service.
- Coordinate with other teams (legal, support, product, etc.) when needed.

Example:

- SEV1 with customer impact → notify leadership
- Possible data breach → involve security immediately
- Cloud provider issue → open a vendor ticket
- Customer communication → involve support team



SENIOR ENGINEER TIP

During incidents, communication is part of the technical response.

- Clear, timely and accurate communication reduces confusion.
- It helps teams make better decisions.
- It builds trust with customers, leadership and other teams.
- Good communication can prevent a small issue from becoming a bigger incident.



EFFECTIVE INCIDENT COMMUNICATION CHECKLIST

- ✓ Severity declared
- ✓ Impact communicated
- ✓ Ownership assigned

- ✓ Facts vs assumptions separated
- ✓ Timeline maintained
- ✓ Regular updates sent

- ✓ Right people involved
- ✓ No speculative root cause
- ✓ Clear and concise language



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

RECOVERY IS NOT THE SAME AS "THE GRAPH IS GREEN"

1 VERIFY CUSTOMER-FACING BEHAVIOR



- Test the actual user journeys and key features.
- Confirm the service is working end-to-end.
- Do not rely only on internal metrics.
- Check from multiple regions if applicable.

Example:

- Log in, place an order, or run a real transaction.
- Confirm the user experience is normal.

2 CONFIRM ERROR RATES NORMALIZE



- Check 4xx and 5xx rates return to normal levels.
- Compare with pre-incident baseline.
- Look at error rates across all services and regions.
- Ensure there are no hidden errors in background jobs or APIs.

Example:

- 5xx rate back to < 0.1% for at least 15 minutes.
- No increase in application or API errors.

3 CONFIRM LATENCY RECOVERY



- Verify response times are back to normal.
- Check p50, p90, and p99 latency.
- Ensure there are no ongoing spikes.
- Monitor both external and internal latency.

Example:

- p95 latency back to baseline (e.g. < 300ms).
- No periodic spikes in the last 15 minutes.

4 CHECK QUEUES AND BACKLOG



- Check message queues, job queues, and event backlogs.
- Ensure the backlog is being processed.
- Watch for queues that continue to grow.
- Confirm background workers are healthy.

Example:

- Queue depth decreasing and returning to normal.
- Old messages are processed successfully.

5 VERIFY REPLICAS AND CAPACITY



- Confirm the correct number of replicas are running.
- Check auto-scaling behaviour.
- Ensure CPU, memory, and other resources are within normal ranges.
- Verify there is no pending or failed scaling action.

Example:

- All expected replicas are healthy and ready.
- No pending scaling events or insufficient resources.

6 LOOK FOR DELAYED FAILURES



- Some issues appear minutes or hours later.
- Watch for recurring errors or latency spikes.
- Check dependent systems for delayed impact.
- Keep monitoring for a reasonable period after the initial recovery.

Example:

- No new error spikes for 30–60 minutes.
- Downstream systems are stable.

7 CONFIRM DEPENDENCIES RECOVERED



- Verify that all external dependencies are healthy.
- Check databases, caches, third-party APIs, and infrastructure services.
- Ensure there are no ongoing connection issues or high error rates.

Example:

- Database latency and error rates are normal.
- Third-party API is responding as expected.

8 WATCH RETRY VOLUME



- Check that retry volume has returned to normal.
- High retries may indicate the problem is not fully resolved.
- Monitor client and service retry metrics.
- Ensure retry storms have stopped.

Example:

- Retry rate back to baseline levels.
- No sustained increase in client or service retries.

9 CHECK DATA INTEGRITY WHERE RELEVANT



- Verify that no data was lost, duplicated, or corrupted.
- Check for incomplete transactions or failed jobs.
- Validate reconciliation processes if needed.
- Confirm critical business data is consistent.

Example:

- All transactions processed successfully.
- No missing or duplicate records.

10 CONTINUE OBSERVATION AFTER MITIGATION



- Keep monitoring key metrics for an extended period (e.g. 30–60 minutes or longer).
- Ensure the system remains stable.
- Confirm there are no secondary issues.
- Close the incident only when you are confident the service is fully recovered.

Example:

- Monitor for at least 30–60 minutes after recovery.
- No new errors, latency spikes, or customer complaints.

11 RECOVERY SEQUENCE



MITIGATE
Take immediate action to stop the impact.



STABILIZE
Ensure the system is steady and functioning.



VERIFY
Confirm all key metrics and behaviour are normal.



OBSERVE
Continue monitoring for a reasonable period.



CLOSE
Confirm recovery and officially close the incident.



SENIOR ENGINEER TIP

Recovery is more than metrics. Always confirm real user behaviour, dependencies, data integrity, and stability over time.

HANDOFF, POST-INCIDENT REVIEW, AND PREVENTING RECURRENCE

1

WHAT HAPPENED



- Brief, factual summary of the incident.
- When it started and when it was resolved.
- Primary symptoms observed.
- Customer and business impact.
- Key contributing factors (if known).

Example:

On 2026-10-01 at 13:24 UTC, the checkout service experienced high error rates, causing failed transactions for some users.

2

CURRENT SYSTEM STATE



- Confirm services are running and healthy.
- Verify key metrics are normal.
- Check that all mitigations are in place.
- Ensure dependent services are stable.
- Validate customer experience.

Example:

- All services are healthy.
- Error rates back to baseline.
- No active user impact.

3

REMAINING RISK



- Identify any residual instability.
- Highlight areas that may still fail.
- Call out temporary mitigations and their limits.
- Note external dependencies that are still at risk.
- Be clear about what could go wrong next.

Example:

- Database is stable but still recovering.
- There is a risk of high latency if traffic increases suddenly.

4

ACTIONS ALREADY PERFORMED



- List all changes and mitigations applied.
- Include who performed the action and when.
- Explain the reason for each change.
- Document the outcome of each action.

Example:

- Scaled API replicas at 13:40 UTC (Alex).
- Applied rate limiting at 13:55 UTC (Priya).

5

EVIDENCE COLLECTED



- Logs and error messages.
- Metrics and dashboards.
- Traces and request samples.
- Configuration changes.
- Screenshots (if useful).
- Timeline of events.

Example:

- Collected application logs, error traces, and metrics screenshots from 13:20–14:10 UTC.

6

OUTSTANDING HYPOTHESES



- List what is still not confirmed.
- Identify alternative possible causes.
- Note what further evidence is needed.
- Be honest about uncertainty.

Example:

- We have not yet confirmed whether the latency spike was caused by the database or a downstream API.

7

TEMPORARY MITIGATIONS



- Document all temporary workarounds.
- Explain why they are needed and their limitations.
- Include expiry or review date.
- Ensure the team knows how to remove them.

Example:

- Rate limiting is enabled at 100 requests/second until the database is fully stable.

8

FOLLOW-UP WORK



- List tasks to permanently fix the root cause.
- Assign owners and due dates.
- Include testing and validation steps.
- Track to completion.

Example:

- Investigate database performance issue and implement a permanent fix (Owner: David, Due: 2026-10-03).

9

BLAMELESS CAUSAL ANALYSIS



- Focus on what happened and why.
- Identify contributing factors across people, process, technology, and environment.
- Avoid blaming individuals.
- Use the analysis to drive real improvements.

Example:

- Increased traffic, missing alerts, and insufficient capacity planning contributed to the incident.

10

IMPROVEMENTS TO PREVENT RECURRENCE



ALERT IMPROVEMENTS

- Better alerts
- Reduce noise
- Detect earlier



RUNBOOK IMPROVEMENTS

- Clearer steps
- Add missing scenarios
- Test regularly



ARCHITECTURE IMPROVEMENTS

- Remove single points of failure
- Improve resilience
- Simplify dependencies



CAPACITY IMPROVEMENTS

- Right sizing
- Auto-scaling
- Load testing



TESTING IMPROVEMENTS

- Pre-deployment tests
- Chaos/failure testing
- Validate rollbacks



AUTOMATION IMPROVEMENTS

- Automate detection
- Automate remediation where safe
- Reduce manual work

11

INCIDENT LESSON



A resolved incident that teaches the organization nothing is likely to return.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

THE SENIOR ENGINEER ON-CALL DECISION FRAMEWORK

1 WHAT IS THE CUSTOMER IMPACT?



- Which customers are affected?
- What features or services are impacted?
- Is the service partially or fully unavailable?
- What is the business impact?

Example:

- Users cannot log in.
- Checkout is failing, impacting revenue.

2 WHAT IS THE BLAST RADIUS?



- Is it a single service or multiple services?
- Single region or multi-region?
- Internal only or external customer impact?
- Are dependencies affected?

Example:

- Impact is limited to the EU region.
- Affects web, API, and database.

3 WHAT CHANGED?



- Review recent deployments.
- Check configuration changes.
- Look for infrastructure, network, or dependency changes.
- Consider both intended and unintended changes.

Example:

- A new version was deployed 10 minutes before the issue.
- A configuration change was applied.

4 WHAT EVIDENCE SUPPORTS THE LEADING HYPOTHESIS?



- Check metrics, logs, traces, and events.
- Look for patterns across different systems.
- Validate before taking action.
- Avoid acting on incomplete or weak evidence.

Example:

- Error rate increased in all API pods at 14:05 UTC.
- Logs show database connection timeouts.

5 IS THE FAILURE GETTING WORSE?



- Are error rates increasing?
- Is latency getting higher?
- Are more services being affected?
- Is the blast radius expanding?

Example:

- Error rate increased from 5% to 25% in 10 minutes.
- Latency is consistently rising.

6 WHAT IS THE SAFEST REVERSIBLE MITIGATION?



- Choose the lowest-risk action that can be easily reversed.
- Consider rollback, feature flag, traffic shift, or scaling.
- Prefer reversible changes over permanent ones.

Example:

- Roll back to the previous version.
- Shift traffic to a healthy region.

7 WHAT COULD MY INTERVENTION BREAK?



- Consider dependencies and downstream services.
- Evaluate side effects and risks.
- Think about data consistency, security, and customer impact.
- Have a rollback plan.

Example:

- Restarting the service may cause connection drops for active users.
- Scaling down may overload other replicas.

8 WHO ELSE NEEDS TO BE INVOLVED?



- Do I need to involve leadership?
- Should security be notified?
- Are vendors or third-party providers affected?
- Do other teams need to support (e.g., database, network, product)?

Example:

- Notify the security team for a potential data exposure.
- Engage the database team for support.

9 HOW WILL I KNOW RECOVERY ACTUALLY WORKED?



- Confirm customer-facing behavior.
- Verify error rates and latency are normal.
- Check queues and backlog.
- Ensure all dependent services are healthy.

Example:

- Login and checkout work from multiple regions.
- Error rate and latency are back to baseline.

10 WHAT MUST CHANGE SO THIS DOES NOT HAPPEN AGAIN?



- Improve alerts and monitoring.
- Update runbooks.
- Fix architecture or capacity gaps.
- Add or improve testing.
- Automate detection or mitigation where possible.

Example:

- Add alert for database connection failures.
- Implement automated rollback on high errors.

DECISION PATH



ALERT

Receive and acknowledge the alert.



VALIDATE

Confirm the issue and remove false positives.



ASSESS IMPACT

Understand customer and business impact.



DEFINE BLAST RADIUS

Identify the full scope of the issue.



CHECK CHANGES + SIGNALS

Look at recent changes and relevant metrics, logs, and events.



FORM HYPOTHESIS

Identify the most likely cause based on evidence.



MITIGATE SAFELY

Apply the safest reversible mitigation.



VERIFY RECOVERY

Confirm the service is working as expected.



MONITOR

Continue to watch for stability and delayed failures.



DOCUMENT + PREVENT RECURRENCE

Record findings and implement improvements.



INCIDENT LESSON

A resolved incident that teaches the organization nothing is likely to return.



SENIOR ENGINEER DECISION MINDSET

- Be systematic.
- Use evidence.
- Reduce risk.
- Communicate clearly.
- Prefer reversible actions.
- Always learn and improve.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta